

The Complete Learning Guide

From Hello World to Production Security
22 Hands-On Chapters • Real Examples • Best Practices

Based on: github.com/sandeepdass007/docker-kubernetes-learning

22 Chapters

Practical Examples

**Security &
Production**

Kubernetes Bridge

TABLE OF CONTENTS

Preface — Getting the Most from This Book

Chapter 1 — Docker Hello World — First Container

Images, Containers, Build & Run

Chapter 2 — Image Layers & Build Cache

How Docker stores and reuses layers

Chapter 3 — Dockerfile Best Practices

Good vs. bad Dockerfile patterns

Chapter 4 — Why Volumes Exist

Container data is ephemeral

Chapter 5 — Container Networking Basics

EXPOSE vs port publishing

Chapter 6 — Docker Networks Deep Dive

Bridge, host, none, user-defined

Chapter 7 — Network Drivers

bridge/host/none comparisons

Chapter 8 — Container-to-Container Communication

DNS, service discovery

Chapter 9 — Docker Compose

Multi-service orchestration

Chapter 10 — Efficient Image Layering

Optimising Dockerfile layer order

Chapter 11 — Multi-Stage Builds

Separate build and runtime

Chapter 12 — Volumes & Persistent Data

Bind mounts, named volumes, tmpfs

Chapter 13 — Environment Variables & Secrets

12-factor app configuration

Chapter 14 — Health Checks & Restart Policies

Self-healing containers

Chapter 15 — Resource Limits

CPU and memory cgroups

Chapter 16 — Container Security

Non-root, caps, seccomp, read-only FS

Chapter 17 — Container Logging

Drivers, log rotation, centralization

Chapter 18 — Advanced Docker Build

BuildKit, secrets, multi-arch, .dockerignore

Chapter 19 — Container Process Model

PID 1, signals, tini

Chapter 20 — Docker Debugging Tools

inspect, stats, exec, events

Chapter 21 — Image Registries

Docker Hub, private registries, tagging

Chapter 22 — Storage Internals

Union FS, overlay2, copy-on-write

Bonus — Docker to Kubernetes — The Bridge

How Docker concepts map to K8s

Preface — Getting the Most from This Book

This guide is a companion to the hands-on learning repository **docker-kubernetes-learning**. It follows the same chapter structure but expands every concept with plain-English explanations, diagrams described in words, real command outputs, comparisons, and production tips that are not in the original README files.

Every chapter starts with **Why does this matter?**, builds up the concept step-by-step, shows runnable examples with expected output, and ends with a **Key Takeaways** box. Important warnings appear in yellow boxes; production best-practices in blue boxes; additional tips in green boxes.

How to Use This Book

- Read linearly if you are a beginner — each chapter builds on the previous one.
- Jump to any chapter if you already know the basics and want a reference.
- Run the examples from the repository while reading each chapter.
- Use the bonus chapter at the end as a bridge into Kubernetes learning.

■ TIP

All examples were tested on Linux and macOS with Docker Desktop and Docker Engine 24+. Windows users should enable WSL2 for best results.

CHAPTER 1

Images • Containers • Build & Run

1.1 What is Docker?

Docker is a platform for packaging applications into lightweight, portable units called **containers**. A container bundles the application code, its runtime, libraries, and configuration into a single artefact that runs identically on any machine that has Docker installed — whether that machine is your laptop, a CI server, or a cloud VM.

Before containers existed, the common reply to 'it works on my machine' was to set up long installation guides or to ship entire virtual machines (VMs). Docker solves this by sharing the host OS kernel and isolating only the application-level dependencies.

The Three-Tier Mental Model

Everything in Docker maps to one of three concepts: **Dockerfile**, **Image**, and **Container**.

Concept	Analogy	What it is
Dockerfile	Recipe / Blueprint	A text file with instructions to build an image
Image	Baked cake / Template	A read-only, layered snapshot of a filesystem
Container	Running slice of cake	A live, running process started from an image

How Docker is Different from a VM

A Virtual Machine (VM) contains a full OS kernel, device drivers, and system libraries in addition to the application. A Docker container shares the host OS kernel and only isolates the user-space — making it start in milliseconds instead of minutes and use megabytes instead of gigabytes.

Feature	Virtual Machine	Docker Container
Start time	30-120 seconds	< 1 second
Size	GBs	MBs
OS	Full guest OS	Shared host kernel
Isolation	Strong (hypervisor)	Process-level (namespaces + cgroups)
Portability	Moderate	Very high
Resource overhead	High	Low

1.2 Installing Docker

The repository ships a helper script **bootstrap.sh** that installs Docker. You can also install Docker Desktop from <https://www.docker.com/get-started>.

Terminal

```
# After installation, verify it works:
docker --version
# Docker version 24.0.5, build ced0996
docker run hello-world
# Hello from Docker!
# This message shows your installation appears to be working correctly.
```

1.3 Your First Container — Chapter 1 Walk-Through

The repository's **chapter-01-hello-world** folder contains a minimal Dockerfile, a **run.sh** script, and a **clean.sh** script. Let us walk through each file.

The Dockerfile

A Dockerfile is just a text file. Each line is an instruction. Here is the simplest possible Dockerfile:

Dockerfile

```
# Start from an official Python base image
FROM python:3.11-slim
# Set the working directory inside the container
WORKDIR /app
# Print a message when the container runs
CMD ["echo", "Hello from Docker! Chapter 1 complete."]
```

Breaking Down Each Instruction

Instruction	What it does
FROM python:3.11-slim	Pulls a pre-built Python image from Docker Hub as the base layer. 'slim' means a minimal version.
WORKDIR /app	Creates /app inside the container and sets it as the default directory for subsequent commands.
CMD [...]	Defines the default command that runs when the container starts. Exec-form (array) is preferred.

Building the Image

Terminal

```
# Run from inside chapter-01-hello-world/
docker build -t hello-docker .
# Flags:
# -t hello-docker → tag (name) the image 'hello-docker'
# . → use the current directory as the build context
```

The **build context** is the folder Docker reads files from during the build. The dot (.) means 'current directory'. Docker reads the Dockerfile in that directory and executes each instruction to produce an image.

Running the Container

Terminal

```
docker run hello-docker
# Hello from Docker! Chapter 1 complete.
# Run interactively with a shell:
docker run -it hello-docker /bin/sh
# Run and remove container when it exits:
docker run --rm hello-docker
```

Essential Docker Commands — Quick Reference

Command	Purpose
docker build -t name .	Build image from Dockerfile in current directory
docker run	Start a container from an image
docker run -it sh	Start an interactive shell inside a container
docker run -d	Start container in detached (background) mode
docker ps	List running containers
docker ps -a	List all containers (including stopped)
docker images	List locally available images
docker stop	Stop a running container gracefully
docker rm	Remove a stopped container
docker rmi	Remove a locally stored image
docker logs	View stdout/stderr output of a container
docker exec -it sh	Enter a running container interactively

■ NOTE

Key Takeaways Chapter 1: • Docker wraps applications and dependencies into portable containers. • A Dockerfile is the recipe; an image is the built artifact; a container is the running process. • Containers share the host OS kernel — they are lightweight and fast. • docker build creates images; docker run starts containers.

CHAPTER 2

How Docker builds and stores images efficiently

2.1 Docker Images are Made of Layers

When Docker builds an image, it executes each instruction in the Dockerfile one by one. Each instruction creates a new **read-only layer** on top of the previous one. Think of it like a stack of transparent sheets — together they form the final image.

Dockerfile (chapter-02)

```
FROM python:3.11-slim # Layer 0 – base image (already cached if pulled before)
WORKDIR /app # Layer 1 – sets working directory metadata
COPY message.txt . # Layer 2 – adds message.txt to the filesystem
RUN cat message.txt # Layer 3 – executes a command (output captured)
CMD ["echo","done"] # Layer 4 – container launch instruction (metadata only)
```

You can inspect every layer of any image using **docker history**:

Terminal

```
docker history hello-docker

IMAGE CREATED CREATED BY SIZE
a1b2c3d4e5f6 2 minutes ago CMD ["echo" "done"] 0B
<missing> 2 minutes ago COPY message.txt . 32B
<missing> 2 minutes ago WORKDIR /app 0B
<missing> 6 weeks ago python:3.11-slim base layers 120MB
```

2.2 The Build Cache

Docker is smart: before executing an instruction it checks whether it has already run the exact same instruction on the exact same input files. If yes, it reuses the existing layer from **cache** instead of rebuilding it. This makes repeated builds very fast.

Build output comparison

```
# First build – everything is fresh (no cache):
Step 1/4 : FROM python:3.11-slim => Already cached
Step 2/4 : WORKDIR /app => Running
Step 3/4 : COPY message.txt . => Running
Step 4/4 : RUN cat message.txt => Running

# Second build – no files changed:
Step 1/4 : FROM python:3.11-slim => Already cached
Step 2/4 : WORKDIR /app => Using cache
Step 3/4 : COPY message.txt . => Using cache
Step 4/4 : RUN cat message.txt => Using cache

# Third build – message.txt was edited:
Step 1/4 : FROM python:3.11-slim => Already cached
Step 2/4 : WORKDIR /app => Using cache
Step 3/4 : COPY message.txt . => Running <-- CACHE MISS
Step 4/4 : RUN cat message.txt => Running <-- must rebuild
```

Cache Invalidation Rule

This is the most important rule: **when a layer is invalidated, all subsequent layers must also be rebuilt**. Docker processes instructions top-to-bottom. Once it encounters a change, it cannot trust

any later cached layer.

■ ■ WARNING

This rule explains why Dockerfile instruction ORDER matters enormously for performance. Always place the instructions that change most often (like copying application code) at the BOTTOM of the Dockerfile.

2.3 Experimenting with the Cache (Step-by-Step)

- **Step 1:** Run `./run.sh`
 - All layers are built fresh — no cache exists yet.
- **Step 2:** Run `./run.sh` (no changes)
 - All layers show 'Using cache'. Build completes instantly.
- **Step 3:** Run `Edit message.txt`, then `./run.sh`
 - COPY layer and everything after it rebuild. Layers before the COPY are still cached.

■ NOTE

Key Takeaways Chapter 2: • Every Dockerfile instruction creates an immutable read-only layer. • Docker caches layers and reuses them unless inputs change. • Changing any layer invalidates ALL layers that come after it. • Instruction order directly affects build speed — put rarely-changing instructions first.

CHAPTER 3

Writing Dockerfiles that are fast, small, and predictable

3.1 The Bad Dockerfile Pattern

Chapter 3 of the repository compares two Dockerfiles for the same Python application. Here is the bad one:

```
Dockerfile.bad
FROM python:3.11
WORKDIR /app
# ■ COPY everything first – including app code, data, and requirements
COPY . .
# ■ Install dependencies AFTER copying code
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "main.py"]
```

The problem: **COPY . .** copies every file in the build context, including application code. If you change one line of **main.py**, the COPY layer is invalidated and Docker re-runs **pip install** even though **requirements.txt** has not changed. A dependency install can take minutes.

3.2 The Good Dockerfile Pattern

```
Dockerfile.good
# ■ Use a slim base image (smaller, fewer CVEs)
FROM python:3.11-slim
WORKDIR /app
# ■ Copy ONLY the dependency file first
COPY requirements.txt .
# ■ Install dependencies – this layer is cached until requirements.txt changes
RUN pip install --no-cache-dir -r requirements.txt
# ■ Copy application code LAST – changes here don't invalidate pip install
COPY app/ ./app/
COPY data/ ./data/
CMD ["python", "app/main.py"]
```

Why This Works

The key insight: **requirements.txt** changes far less often than application code. By copying it first and installing dependencies immediately, we let Docker cache the heavy pip install step. Only **when requirements.txt changes** does Docker re-run pip install.

Scenario	Bad Dockerfile	Good Dockerfile
Change main.py	pip install re-runs	pip install cached ■
Add new dependency	pip install re-runs	pip install re-runs (correct)
Change data file	pip install re-runs	pip install cached ■
First build	Full install	Full install (same)

3.3 Additional Dockerfile Best Practices

✓ Use slim or alpine base images

python:3.11-slim is ~120 MB vs python:3.11 at ~900 MB. Smaller images pull faster, have a smaller attack surface, and cost less storage.

✓ Pin dependency versions

requirements.txt should specify exact versions (e.g. flask==3.0.2) so builds are reproducible. Floating versions can break silently.

✓ Combine RUN commands to reduce layers

RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/* is one layer. Separate RUN commands would leave the apt cache in earlier layers even after deletion.

✓ Use .dockerignore

A .dockerignore file prevents large or sensitive files (node_modules, .git, .env) from being sent to the Docker daemon as build context.

✓ Run as non-root

Add USER appuser before CMD. See Chapter 16 for the full security discussion.

✓ Set explicit WORKDIR

Never rely on implicit /. Always set WORKDIR so paths are predictable.

.dockerignore + combined RUN

```
# Example of combining apt commands to keep layers small:
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
# Example .dockerignore:
.git
node_modules
__pycache__
*.pyc
.env
data/large_dump.csv
```

■ NOTE

Key Takeaways Chapter 3: • Copy dependency files first, install, then copy application code. • Use slim base images to reduce size and attack surface. • Combine shell commands in a single RUN to minimise layers. • Use .dockerignore to exclude unnecessary build context files.

CHAPTER 4

Containers are ephemeral — but data doesn't have to be

4.1 Containers are Ephemeral by Design

A container is just a process. When the process exits, the container stops. When the container is removed, its writable filesystem layer is destroyed. This is not a bug — it is a fundamental design principle.

The consequence: **any data written inside a container's filesystem is lost when that container is removed.**

Demonstrating data loss

```
# Step 1: Write data inside a container
docker run --rm python:3.11-slim sh -c 'echo hello > /data.txt && cat /data.txt'
# hello

# Step 2: Run a new container from the same image
docker run --rm python:3.11-slim cat /data.txt
# cat: /data.txt: No such file or directory

# The data is GONE — it never left the first container's writable layer
```

4.2 The Lifecycle Mismatch

Applications often need data that outlives any single container instance. Databases, file uploads, configuration, logs — none of these should be tied to the lifecycle of a container.

What has its own lifecycle	Container data	What we want
Application code	Same as container	Rebuild on release
Database records	Same as container	Survive container restarts
User uploads	Same as container	Survive container restarts
Configuration	Same as container	Injected at runtime

4.3 Docker's Solution: Volumes

Docker Volumes decouple data storage from container lifecycle. A volume is a directory that exists **outside** of the container's writable layer — managed either by Docker (named volumes) or by the host filesystem (bind mounts). Containers can be removed and recreated while the volume persists.

Named volume lifecycle

```
# Create a named volume and attach it to a container:
docker volume create mydata
docker run -v mydata:/app/data myimage

# Even after removing the container, the volume survives:
docker rm my_container
docker volume ls
# DRIVER VOLUME NAME
# local mydata ← still here!
```

■ NOTE

Key Takeaways Chapter 4: • Containers are ephemeral — removing a container destroys its filesystem. • Never store important data only inside a container's writable layer. • Volumes exist outside the container lifecycle. • Volume types are covered in depth in Chapter 12.

CHAPTER 5

EXPOSE vs port publishing — making services reachable

5.1 The Isolation Problem

Every Docker container has its own **network namespace** — a private, isolated network stack with its own interfaces, routing table, and port space. A service listening on port 8000 *inside* the container is **not automatically reachable** from outside the container, even from the host machine.

What EXPOSE Actually Does

The EXPOSE instruction in a Dockerfile is purely **documentation**. It tells humans and tools which port the application uses, but it does NOT publish that port to the host. Think of EXPOSE as a label, not a firewall rule.

EXPOSE alone is not enough

```
# EXPOSE is documentation only – it does NOT publish the port
EXPOSE 8000

# This container runs, but localhost:8000 is UNREACHABLE from host:
docker run myapp # no -p flag

# curl will time out:
curl http://localhost:8000

# curl: (7) Failed to connect to localhost port 8000: Connection refused
```

5.2 Publishing Ports with -p

To make a container's port reachable from the host, use the **-p** flag when running the container. It creates a NAT rule that forwards traffic from a host port to the container port.

Publishing a port

```
# Syntax: docker run -p <host_port>:<container_port> <image>
docker run -p 8080:8000 myapp

# Now the service is reachable:
curl http://localhost:8080

# Hello from inside the container!

# Port 8000 is the container port (what the app listens on)
# Port 8080 is the host port (what you connect to from outside)
```


■ NOTE

Key Takeaways Chapter 5: • Containers run in their own network namespace — they are isolated. • EXPOSE is documentation only. It does NOT open any port. • Use `docker run -p host_port:container_port` to make a service reachable. • The host port and container port can be different numbers.

CHAPTER 6

Bridge • Host • None • User-Defined

6.1 Why localhost Doesn't Work Between Containers

Each container has its own network namespace. **localhost** always refers to the network namespace of the container that calls it — never to another container. So if Container A does `requests.get('http://localhost:5000')`, it is calling port 5000 inside Container A, not Container B.

The localhost trap

```
# Container A (client) tries to reach Container B (server) at localhost:5000
# This ALWAYS fails – localhost is A's loopback, not B's
# Error seen:
# requests.exceptions.ConnectionError: HTTPConnectionPool(
# host='localhost', port=5000): Connection refused
```

6.2 Docker Network Types

Network Type	Use Case	Container Isolation	DNS by Name
bridge (default)	Single host, multiple containers	Yes	No
user-defined bridge	Same host, service discovery	Yes	Yes ■
host	Max performance, shares host IP	No	No
none	Full isolation, no network	Yes	No
overlay	Multi-host (Docker Swarm)	Yes	Yes ■

Default Bridge Network

When you run a container without specifying a network, it joins the default **docker0** bridge. Containers on the default bridge can communicate by IP, but **not by name**. Docker does not provide DNS resolution on the default bridge.

Default bridge – no DNS

```
docker run -d --name server myimage
docker run -d --name client myimage
# Client tries to reach server by name – FAILS on default bridge:
# socket.gaierror: [Errno -2] Name or service not known
```

User-Defined Bridge Network

Create a custom network with `docker network create`. Containers on the same user-defined bridge can resolve each other **by container name** thanks to Docker's embedded DNS server.

User-defined bridge with DNS

```
# Create a custom network
docker network create my-net

# Start both containers on the same network
docker run -d --name server --network my-net myimage
docker run -d --name client --network my-net myimage

# Now the client can reach the server by name:
# requests.get('http://server:5000') ← works!
```

Host Network Mode

With `--network host`, the container shares the host's network stack completely. The container's processes see the same IP addresses, interfaces, and ports as the host. Port mapping (`-p`) is ignored.

Host network

```
docker run --network host myimage

# The app listens on :5000, reachable directly at localhost:5000 on host
# No port mapping needed – and no isolation either
```

WARNING

Host networking bypasses container network isolation. Use only when performance is critical (e.g., high-throughput networking applications) and you understand the security implications.

None Network Mode

With `--network none`, the container has no network interface at all — not even a loopback. It cannot send or receive any network traffic. Use this for batch processing jobs that need maximum isolation.

None network

```
docker run --network none myimage

# Container runs with no network stack
# curl http://anything → fails (no interfaces)
```

NOTE

Key Takeaways Chapter 6: • `localhost` never crosses container boundaries. • Default bridge: containers can communicate by IP but not by name. • User-defined bridge: Docker DNS resolves container names — the recommended approach. • `host` mode: no isolation, maximum performance. • `none` mode: complete network isolation.

CHAPTER 7

Bridge • Host • None — deep dives

CHAPTER 8

DNS-based service discovery in practice

8.1 The Isolated Scenario (Failure)

When two containers are started without a shared network, they cannot reach each other even though they are on the same host machine. This is Docker's default security posture — **isolation by default**.

Isolated containers – failure

```
docker run -d --name server myapp # default bridge
docker run -d --name client myapp # default bridge

# Client attempts: requests.get('http://server:5000')
# → socket.gaierror: Name or service not known
# (DNS does not work on default bridge)
```

8.2 The Working Scenario (Same Network)

Attaching both containers to the same user-defined bridge gives them DNS-based discovery. The client can reference the server using its **container name** as the hostname.

Same network – success

```
docker network create comm-net
docker run -d --name server --network comm-net myapp
docker run -d --name client --network comm-net myapp

# Client attempts: requests.get('http://server:5000')
# → 200 OK: 'Hello from server!'

# How Docker resolves 'server':
# Docker runs an embedded DNS server at 127.0.0.11
# When client resolves 'server', Docker DNS returns the server container's IP
```

How Docker DNS Works

Docker's embedded DNS server (127.0.0.11) listens inside every container on a user-defined network. When a container queries a hostname (e.g. 'server'), the DNS server looks up which container has that name on the shared network and returns its private IP address. This is why container names work as hostnames.

■ NOTE

Key Takeaways Chapter 8: • Containers on different networks cannot communicate. • Containers on the same user-defined bridge can use each other's names as hostnames. • Docker runs an embedded DNS server (127.0.0.11) in every user-defined network. • This is the basis for service discovery in Docker Compose.

CHAPTER 9

Defining and running multi-container applications

9.1 The Problem with Manual docker run

Running a multi-service application (server + client + database) manually requires remembering the exact order of commands, network creation, port mappings, and volume mounts. This is error-prone and impossible to reproduce reliably.

Manual orchestration – painful

```
# Manual approach – brittle and easy to forget:
docker network create app-net
docker run -d --name server --network app-net -p 5001:5000 server-image
docker run -d --name client --network app-net client-image
# Forgot to mount the volume?
# Services start in wrong order?
# Good luck on a new machine.
```

9.2 Docker Compose to the Rescue

Docker Compose lets you describe the entire multi-service application in a single YAML file (**docker-compose.yml**) and start everything with one command. Compose automatically creates a shared network, resolves service names, and handles startup ordering.

docker-compose.yml

```
# docker-compose.yml
version: '3.8'

services:
  server:
    build: ./server
    ports:
      - '5001:5000'
  client:
    build: ./client
    depends_on:
      - server
  environment:
    - SERVER_URL=http://server:5000
# One command to start everything:
docker compose up --build
# One command to stop everything:
docker compose down
```

What Docker Compose Does Automatically

- Creates a project-scoped network (e.g. myproject_default).
- Attaches all services to that network.
- Uses service names as DNS hostnames (e.g. the client reaches server at http://server:5000).
- Starts services in dependency order (depends_on).
- Tears down all containers, networks, and anonymous volumes on docker compose down.

9.3 Essential Compose Commands

Compose CLI

```
docker compose up # Start all services
docker compose up --build # Rebuild images then start
docker compose up -d # Start in detached mode
docker compose down # Stop and remove containers + network
docker compose down -v # Also remove named volumes
docker compose logs # View logs from all services
docker compose logs -f server # Follow logs from one service
docker compose ps # List service status
docker compose exec server sh # Shell into a running service
docker compose restart server # Restart one service
```

Common Compose Fields

Field	Purpose	Example
build	Build image from Dockerfile	build: ./server
image	Use existing image	image: postgres:15
ports	Publish ports host:container	- '8080:80'
volumes	Mount volumes	- mydata:/data
environment	Set env variables	- DB_HOST=postgres
env_file	Load env from file	- .env
depends_on	Start order	- db
networks	Custom networks	- my-net
restart	Restart policy	always

■ NOTE

Key Takeaways Chapter 9: • Docker Compose defines multi-service apps as a single YAML file. • All services in a Compose file share an auto-created network and can resolve each other by service name. • `docker compose up --build` starts everything; `docker compose down` tears it all down. • `depends_on` controls startup order but does NOT wait for the service to be ready (use healthchecks for that).

CHAPTER 10

Structuring Dockerfiles for maximum cache efficiency

10.1 The Problem with COPY ..

bad-layering/Dockerfile

```
# Bad layering – COPY .. before pip install:
FROM python:3.11-slim
WORKDIR /app
COPY .. # copies EVERYTHING
RUN pip install -r requirements.txt # runs every time any file changes
CMD ["python", "app.py"]
```

10.2 The Correct Pattern

good-layering/Dockerfile

```
# Good layering – install dependencies first:
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt . # only the dependency file
RUN pip install -r requirements.txt # cached until requirements.txt changes
COPY app.py . # app code – changes frequently
CMD ["python", "app.py"]
```

The principle is simple: **stable things first, volatile things last**. requirements.txt rarely changes. app.py changes constantly. By placing them in separate layers in the right order, the expensive pip install step is almost always served from cache.

■ NOTE

Key Takeaways Chapter 10: • Copy files that change rarely (requirements.txt) before files that change often (app code). • A single changed file invalidates all layers that come after the COPY that includes it. • Good layering can reduce CI/CD build times from minutes to seconds.

CHAPTER 11

Separating build-time and runtime environments

11.1 The Problem with Single-Stage Builds

A single-stage Dockerfile installs build tools, compiles code, and runs the application all in one image. The final image ships with compilers, build scripts, pip caches, and temporary files that are never needed at runtime. This increases image size and attack surface.

bad-single-stage/Dockerfile

```
# Single-stage – build tools end up in the final image:
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt # pip + cache stays forever
CMD ["python", "app.py"]
# Final image: ~950MB (includes all build tooling)
```

11.2 Multi-Stage Builds — The Solution

Each **FROM** instruction in a Dockerfile starts a new build stage. Stages can be named with AS name. You can selectively **COPY --from=name** files from any previous stage. The final stage becomes the runtime image; all other stages are discarded.

good-multi-stage/Dockerfile

```
# Stage 1 – Builder: install dependencies
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2 – Runtime: copy only installed packages + app code
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /install /usr/local # only the installed packages
COPY app.py .
CMD ["python", "app.py"]
# Final image: ~120MB (no build tools, no pip cache)
```

What Gets Discarded

Content	Builder stage	Final runtime image
Application code	■ Present	■ Present
Runtime dependencies	■ Present	■ Present (copied)
pip itself	■ Present	■ Not present
pip cache	■ Present	■ Not present
Build tools	■ Present	■ Not present
Temp / compilation files	■ Present	■ Not present

11.3 Building a Specific Stage (--target)

During development or CI testing you often only need to build up to a certain stage, not the full production image. Use `--target` to stop at any named stage.

Using `--target`

```
# Build only the builder stage (good for debugging dependencies):
docker build --target builder -t myapp-builder .

# Build the full production image:
docker build -t myapp:v1 .

# Run tests inside the builder stage:
docker run --rm myapp-builder python -m pytest tests/
```

■ NOTE

Key Takeaways Chapter 11: • Multi-stage builds use multiple `FROM` instructions in one Dockerfile. • Each stage can be named (AS name) and copied from (`COPY --from=name`). • Only the final stage becomes the runtime image — all others are discarded. • Result: smaller, cleaner, more secure production images. • Use `--target` to build only a specific stage.

CHAPTER 12

Bind mounts • Named volumes • tmpfs

12.1 Overview of Volume Types

Type	Stored on	Docker managed?	Persists after rm?	Best for
No volume	Container layer	No	No	Read-only or throwaway containers
Bind mount	Host filesystem	No	Yes	Local dev, hot-reload
Named volume	Docker directory	Yes	Yes	Databases, production data
tmpfs	RAM (memory)	N/A	No	Secrets, fast temp data

12.2 No Volume — Ephemeral Data

No volume

```
docker run --rm myapp
# App writes /app/data/output.txt inside the container
# Container exits – output.txt is GONE
docker run --rm myapp cat /app/data/output.txt
# cat: /app/data/output.txt: No such file or directory
```

12.3 Bind Mount

A bind mount maps a specific **host directory** into the container. Files written by the container appear on the host, and files on the host are immediately visible inside the container. Perfect for development (live reload).

Bind mount

```
# Syntax: -v /host/path:/container/path
docker run -p 5001:5001 \
-v $(pwd)/host-data:/app/data \
myapp
# The container writes to /app/data/message.txt
# You can read it on the host at ./host-data/message.txt
cat ./host-data/message.txt
# Written at 2026-03-15 14:22:31
# Stop + remove container – host-data/ still exists!
docker rm -f myapp
cat ./host-data/message.txt
# Written at 2026-03-15 14:22:31 ← still there
```

■■ WARNING

Bind mounts give containers write access to your host filesystem. Never bind-mount sensitive directories (`/`, `/etc`, `~/ssh`) into containers.

12.4 Named Volume

A named volume is **managed entirely by Docker**. You reference it by name; Docker decides where to store it on the host (`/var/lib/docker/volumes/`). The volume persists even after removing the container.

Named volume lifecycle

```
# Create a named volume:
docker volume create mydata

# Use it in a container:
docker run -d -p 5001:5001 -v mydata:/data myapp --name myapp1

# Stop + REMOVE container:
docker rm -f myapp1

# Start a NEW container with the same volume:
docker run -d -p 5001:5001 -v mydata:/data myapp --name myapp2

# Data is still there – volume survived container deletion

# Clean up a named volume:
docker volume rm mydata

# List all volumes:
docker volume ls
```

12.5 tmpfs

A **tmpfs** mount is backed by RAM (not disk). It is ultra-fast and automatically destroyed when the container stops. Use it for highly sensitive data (secrets generated at runtime) or intermediate processing that must never be written to disk.

tmpfs

```
docker run --rm \
-p 5001:5001 \
--tmpfs /data \
myapp

# /data is in RAM – fastest possible I/O
# Container stops → /data is gone (RAM freed)
```

■ NOTE

Key Takeaways Chapter 12: • Bind mounts: host path mapped into container — good for dev, risky in prod. • Named volumes: Docker-managed — the production standard for databases and state. • tmpfs: RAM-backed, ultra-fast, ephemeral — for secrets or high-speed temp data. • NEVER store important data only inside a container's own filesystem.

CHAPTER 13

The 12-Factor App way to configure containers

13.1 Why Not Hardcode Configuration?

Hardcoding database hostnames, API keys, or environment names inside source code means you need to rebuild the Docker image every time you want to deploy to a different environment. It also risks committing secrets to version control.

app.py – hardcoded (bad)

```
# ■ HARDCODED – requires rebuild to change
APP_ENV = 'production'
DB_HOST = 'db.internal.company.com'
API_KEY = 'abc123secret'
```

13.2 Environment Variables with -e

Using -e flag

```
# ■ Read from environment at runtime
import os
APP_ENV = os.environ.get('APP_ENV', 'development')
DB_HOST = os.environ.get('DB_HOST', 'localhost')

# Pass at docker run time:
docker run -e APP_ENV=production -e DB_HOST=db.prod.com myapp
docker run -e APP_ENV=staging -e DB_HOST=db.stg.com myapp
# Same image, different configuration!
```

13.3 .env Files

.env file pattern

```
# .env file – keeps the docker run command clean:
APP_ENV=production
DB_HOST=db.prod.com
LOG_LEVEL=INFO

# docker run loads the file:
docker run --env-file .env myapp

# In docker-compose.yml:
services:
  app:
    env_file:
      - .env
```

■ ■ WARNING

Add .env to your .gitignore. Commit a .env.example with empty values so other developers know what variables are needed.

13.4 Secrets — File-Based Approach

Environment variables appear in docker inspect output and process listings — not suitable for passwords or tokens. Docker Secrets (and Kubernetes Secrets) mount sensitive data as files

inside the container at `/run/secrets/`.

Docker secrets

```
# secrets/db_password.txt:
super-secret-password

# docker-compose.yml:
services:
  app:
    secrets:
      - db_password
    secrets:
      db_password:
        file: ./secrets/db_password.txt

# In Python, read the secret from file:
with open('/run/secrets/db_password') as f:
    db_password = f.read().strip()
```

Method	Visible in docker inspect?	Secure?	Best for
Hardcoded	In image layers	■ No	Nothing — avoid
-e flag	Yes	Partial	Non-sensitive config
--env-file	Yes	Partial	Non-sensitive config
Docker Secrets	No	■ Yes	Passwords, tokens, certs

■ NOTE

Key Takeaways Chapter 13: • Never hardcode configuration — use environment variables. • Pass config via -e flags or --env-file for non-sensitive values. • Use Docker Secrets (file-based) for passwords and tokens. • In Kubernetes: env vars become ConfigMaps; secrets become Kubernetes Secrets.

CHAPTER 14

Self-healing containers and liveness detection

14.1 Running Does Not Mean Healthy

A container can show status **Up** while the application inside is completely broken — stuck in a deadlock, unable to accept connections, or returning errors. Health checks let Docker periodically verify that the application is actually functioning, not just running.

14.2 Defining a HEALTHCHECK in a Dockerfile

Dockerfile with HEALTHCHECK

```
FROM python:3.11-slim
# Install curl so healthcheck command works:
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY app.py .
RUN pip install flask
HEALTHCHECK \
  --interval=10s \
  --timeout=3s \
  --retries=3 \
  CMD curl -f http://localhost:5001/health || exit 1
CMD ["python", "app.py"]
```

Parameter	Meaning	Default
--interval	How often to run the check	30s
--timeout	Max time for check to complete	30s
--retries	Failures before marking unhealthy	3
--start-period	Grace period after container starts	0s

Observing Health Status

Monitoring health

```
docker ps
# CONTAINER ID IMAGE STATUS
# a1b2c3 myapp Up 30 seconds (healthy)
# After the /health endpoint starts failing:
# a1b2c3 myapp Up 2 minutes (unhealthy)
# Detailed health check history:
docker inspect myapp | grep -A 20 Health
```

14.3 Restart Policies

Restart policies tell Docker what to do when a container exits unexpectedly. They are Docker's simplest self-healing mechanism.

Policy	Behaviour
no (default)	Never restart
on-failure[:n]	Restart only if exit code is non-zero; optional max retries
always	Always restart regardless of exit code
unless-stopped	Restart unless the user manually stopped it

Restart policy

```
# Run a container that always restarts on crash:
docker run -d --restart=always --name webapp myapp

# Check restart count after a few crashes:
docker inspect webapp | grep RestartCount
# "RestartCount": 3

# Follow logs to watch the cycle:
docker logs -f webapp
```

■ NOTE

Key Takeaways Chapter 14: • Running does not mean healthy — always define a HEALTHCHECK for production services. • HEALTHCHECK runs a command periodically and marks the container healthy or unhealthy. • Restart policies enable basic self-healing when processes crash. • In Kubernetes: HEALTHCHECK maps to Liveness Probe and Readiness Probe.

CHAPTER 15

CPU and memory constraints with Linux cgroups

15.1 Why Limits Matter

Without resource limits, a single misbehaving container can consume all available CPU or RAM on a host, starving other containers and potentially crashing the machine. Docker uses Linux **cgroups** (control groups) to enforce hard limits on resources.

15.2 CPU Limits

CPU limits

```
# Limit to 0.5 CPU cores (50% of one core):
docker run -d --cpus='0.5' --name cpu-demo myapp

# Limit to 2 cores:
docker run -d --cpus='2' myapp

# Monitor CPU usage in real time:
docker stats

# CONTAINER CPU % MEM USAGE / LIMIT
# cpu-demo ~50% 28MiB / 1.93GiB
```

The `--cpus` flag maps to the Linux CFS (Completely Fair Scheduler) quota. Docker calculates the appropriate `cpu_quota` and `cpu_period` values for the kernel automatically.

15.3 Memory Limits

Memory limits

```
# Limit container to 100MB RAM:
docker run -d --memory='100m' --name mem-demo myapp

# When the container exceeds the limit, the kernel OOM-kills it.
# Exit code 137 indicates an OOM kill:
docker ps -a

# CONTAINER STATUS
# mem-demo Exited (137) 2 minutes ago

# Confirm OOM kill:
docker inspect mem-demo | grep OOMKilled

# "OOMKilled": true
```

Flag	Purpose	Example
<code>--cpus</code>	CPU core limit (fractions allowed)	<code>--cpus='0.5'</code>
<code>--cpu-shares</code>	Relative CPU weight (default 1024)	<code>--cpu-shares=512</code>
<code>--memory</code>	Hard RAM limit	<code>--memory='256m'</code>
<code>--memory-swap</code>	RAM + swap limit	<code>--memory-swap='512m'</code>
<code>--memory-reservation</code>	Soft RAM limit	<code>--memory-reservation='100m'</code>

■ TIP

In Kubernetes, these map to `resources.limits.cpu` and `resources.limits.memory` in the pod spec. Always set resource limits for production workloads.

■ NOTE

Key Takeaways Chapter 15: • Without limits, one container can consume all host resources. • `--cpus` limits CPU time; `--memory` limits RAM. • OOM kill (exit 137) means the container exceeded its memory limit. • These map directly to Kubernetes resource limits.

CHAPTER 16

Non-root • Read-only FS • Capabilities • Seccomp

16.1 Defence in Depth

Container security is not a single feature — it is a layered set of controls. The repository covers five complementary techniques. Used together they significantly reduce the blast radius of any container compromise.

Technique	Protects Against	Docker Flag / Instruction
Non-root user	Root exploits inside container	USER in Dockerfile
Read-only filesystem	File modification / malware persistence	--read-only
Drop capabilities	Unnecessary kernel privileges	--cap-drop ALL
no-new-privileges	Privilege escalation via setuid	--security-opt no-new-privileges
Seccomp profile	Dangerous kernel syscalls	--security-opt seccomp=profile.json

16.2 Running as Non-Root

Non-root container

```
# In Dockerfile – create a dedicated user:
FROM python:3.11-slim
WORKDIR /app
COPY app.py .

# Create a non-root user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# Switch to that user
USER appuser

CMD ["python", "app.py"]

# Verify inside container:
docker exec myapp whoami

# appuser (not root!)
```

16.3 Read-Only Filesystem

Read-only filesystem

```
# Force the container root filesystem to be read-only:
docker run --read-only myapp

# Any write attempt fails:
# PermissionError: [Errno 30] Read-only file system: '/app/output.txt'

# If the app needs writable temp space, add a tmpfs mount:
docker run --read-only --tmpfs /tmp myapp
```

16.4 Dropping Linux Capabilities

Linux capabilities break the all-or-nothing root privilege model into fine-grained permissions. Docker grants a default set of capabilities. Production containers should drop all capabilities and

add back only those strictly needed.

Capabilities

```
# Drop ALL capabilities (maximum restriction):
docker run --cap-drop ALL myapp

# Drop all, add back only NET_BIND_SERVICE (bind to ports < 1024):
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp

# Inspect capabilities on a running container:
docker inspect myapp | grep -i capdrop
```

16.5 no-new-privileges

no-new-privileges

```
# Prevent privilege escalation via setuid binaries:
docker run --security-opt no-new-privileges myapp

# Without this flag:
# Before: UID 1001, After setuid exploit: UID 0 (root) ← BAD

# With this flag:
# Before: UID 1001, After setuid exploit: UID 1001 ← Protected
```

16.6 Seccomp Profiles

Seccomp (Secure Computing Mode) restricts which Linux kernel system calls a container process may invoke. Docker applies a default seccomp profile that blocks ~44 dangerous syscalls. You can create custom profiles to be even more restrictive.

Custom seccomp profile

```
# Custom seccomp profile (seccomp-deny-clone.json):
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "syscalls": [
    {
      "names": ["clone"],
      "action": "SCMP_ACT_ERRNO"
    }
  ]
}

# Apply the profile:
docker run --security-opt seccomp=seccomp-deny-clone.json myapp

# Result: os.fork() raises OSError: Operation not permitted
```

■ NOTE

Key Takeaways Chapter 16: • Always run as a non-root user in production. • Use `--read-only` for stateless services; add `--tmpfs` for required writable paths. • Drop all capabilities (`--cap-drop ALL`) and add back only what is needed. • Enable `--security-opt no-new-privileges` to block `setuid` escalation. • Seccomp provides kernel-level syscall filtering for maximum security.

CHAPTER 17

stdout/stderr • Logging drivers • Log rotation

17.1 The Golden Rule: Log to stdout/stderr

Containers are ephemeral. Writing log files inside a container is problematic: logs are lost when the container is removed, they consume writable layer space, and they cannot be centrally collected. The correct pattern is to write all logs to **stdout** (normal output) and **stderr** (error output). Docker captures these streams and routes them through a configurable **logging driver**.

Logging to stdout

```
# Correct: write to stdout/stderr
import sys, time
while True:
    print('[INFO] Processing job...') # → stdout
    print('[ERROR] Failed!', file=sys.stderr) # → stderr
    time.sleep(2)

# Never do this in a container:
# logging.basicConfig(filename='/app/app.log') # ← writes to container filesystem
```

17.2 Reading Container Logs

docker logs commands

```
docker logs myapp # all logs
docker logs -f myapp # follow in real time (like tail -f)
docker logs --tail 20 myapp # last 20 lines
docker logs -t myapp # add timestamps
docker logs --since 5m myapp # logs from last 5 minutes
```

17.3 Loading Drivers

Driver	What it does	Use case
json-file (default)	Stores JSON logs on host filesystem	Development, simple setups
local	Compact binary format, auto-rotation	Production standalone hosts
none	Discards all logs	External log agents handle logging
fluentd	Forwards to Fluentd collector	Centralised logging pipeline
awslogs	Sends to AWS CloudWatch	AWS deployments
gcplogs	Sends to Google Cloud Logging	GCP deployments

Where json-file Driver Stores Logs

json-file log format

```
# Default log location on Linux host:
/var/lib/docker/containers/<container-id>/<container-id>-json.log

# Each line is a JSON object:
{"log":"Processing job #1\n","stream":"stdout","time":"2026-03-14T14:35:20Z"}
{"log":"Error occurred\n","stream":"stderr","time":"2026-03-14T14:35:21Z"}
```

17.4 Log Rotation

Without rotation, log files grow indefinitely and can fill the entire disk, causing containers and the Docker daemon to fail. Always configure log rotation in production.

Log rotation config

```
# Per container (in docker run):
docker run -d \
  --log-driver json-file \
  --log-opt max-size=10m \
  --log-opt max-file=5 \
  myapp

# Globally for all containers (/etc/docker/daemon.json):
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "5"
  }
}

# Reload Docker daemon after changing daemon.json:
sudo systemctl reload docker
```

■ NOTE

Key Takeaways Chapter 17: • Always log to stdout/stderr — never write log files inside a container. • Docker captures stdout/stderr via logging drivers. • Default driver is json-file; local is more efficient for high-volume logs. • Always configure max-size and max-file to prevent disk exhaustion.

CHAPTER 18

BuildKit • ARG vs ENV • Build secrets • Multi-arch • .dockerignore

18.1 ARG vs ENV

Feature	ARG	ENV
Available during build	■ Yes	■ Yes
Available at runtime (container)	■ No	■ Yes
Override with docker build	■ Yes (--build-arg)	■ No
Override with docker run	■ No	■ Yes (-e)
Visible in docker history	■■ Yes	■ Yes
Best for	Build-time config (versions)	Runtime config (env name)

ARG and ENV interaction

```

ARG APP_VERSION=1.0 # Build-time only – gone after build
ENV APP_ENV=development # Runtime too – visible in container
# Convert ARG to ENV so it survives into the runtime:
ARG APP_VERSION
ENV APP_VERSION=$APP_VERSION
# Build with custom arg:
docker build --build-arg APP_VERSION=2.5 -t myapp .
# Runtime override:
docker run -e APP_ENV=production myapp

```

18.2 Build Cache Behaviour

Every COPY, RUN, and ADD instruction creates a cache key based on the instruction text and (for COPY/ADD) the file hash. If either changes, the cache is busted for that layer and all subsequent layers.

Cache control

```

# Force cache bust for all layers after this point:
ARG CACHEBUST=1
RUN echo $CACHEBUST > /dev/null
# Disable cache for a single build:
docker build --no-cache -t myapp .

```

18.3 Docker Build Secrets (BuildKit)

Never pass secrets via ARG or ENV — they appear in image history and docker inspect. Docker BuildKit supports **build-time secrets** that are mounted temporarily during a RUN command and never baked into any layer.

BuildKit secrets

```
# Enable BuildKit:
export DOCKER_BUILDKIT=1

# In Dockerfile – access secret during RUN:
# syntax=docker/dockerfile:1
FROM python:3.11-slim
RUN --mount=type=secret,id=mysecret \
cat /run/secrets/mysecret && pip install my-private-package

# Build with secret file:
docker build --secret id=mysecret,src=./secret.txt -t myapp .

# Verify secret is NOT in image:
docker history myapp # secret does not appear
```

18.4 .dockerignore

.dockerignore

```
# .dockerignore – exclude from build context sent to Docker daemon
.git
node_modules
__pycache__
*.pyc
.env
*.log
tests/
docs/
large-dummy-file.bin

# Without .dockerignore:
# Sending build context to Docker daemon 250MB

# With .dockerignore:
# Sending build context to Docker daemon 8.5kB
```

18.5 Multi-Architecture Builds with Buildx

Multi-arch with Buildx

```
# Create a Buildx builder that supports multiple platforms:
docker buildx create --use

# Build for AMD64 + ARM64 and push to registry:
docker buildx build \
--platform linux/amd64,linux/arm64 \
-t username/myapp:v1 \
--push \
.

# Docker automatically selects the right arch when pulling:
docker pull username/myapp:v1 # gets amd64 on x86, arm64 on Apple Silicon
```

■ NOTE

Key Takeaways Chapter 18: • ARG is build-time only; ENV is available at runtime too. • Use BuildKit secrets (`--mount=type=secret`) for sensitive build-time data. • Always create a `.dockerignore` to keep build context small. • Use `docker buildx` to create images for multiple CPU architectures.

CHAPTER 19

PID 1 • Shell form vs exec form • tini

19.1 A Container IS a Process

A Docker container is not a virtual machine. It is a single Linux process (or a small group of processes) running in an isolated namespace. The first process that starts inside the container becomes **PID 1**.

In Linux, PID 1 is special: it receives signals from the kernel (SIGTERM, SIGKILL), it is responsible for **reaping zombie child processes**, and if it exits the entire container stops.

19.2 Shell Form vs Exec Form CMD

Shell form vs exec form

```
# Shell form – Docker wraps the command in /bin/sh -c
CMD python app.py
# What Docker actually runs: /bin/sh -c 'python app.py'
# Process tree: PID 1 = /bin/sh, PID 7 = python
# SIGTERM goes to /bin/sh, NOT to python → slow/broken shutdown

# Exec form – Docker runs the command directly
CMD ["python", "app.py"]
# Process tree: PID 1 = python
# SIGTERM goes directly to python → clean shutdown
```

Aspect	Shell form CMD	Exec form CMD
PID 1	/bin/sh	Your app directly
Signal delivery	To /bin/sh only	To your app ■
Zombie reaping	May fail	Works if app handles it
Variable expansion	Yes (shell)	No (literal args)
Recommended	■ No	■ Yes

19.3 Using tini as Init System

tini is a tiny init system designed specifically for containers. It wraps your application, properly forwards signals to it, and reaps zombie child processes. It is the recommended approach for production containers that spawn child processes.

tini in production

```
# Method 1 – Use Docker's built-in init (easiest):
docker run --init myapp

# Method 2 – Install tini in Dockerfile:
FROM python:3.11-slim
RUN apt-get update && apt-get install -y tini && rm -rf /var/lib/apt/lists/*
ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["python", "app.py"]

# Process tree with tini:
# PID 1: tini
# PID 7: python app.py ← tini forwards signals here
```

■ NOTE

Key Takeaways Chapter 19: • A container IS a process — PID 1 controls the container lifecycle. • Always use exec form CMD (["...", "..."]) — it makes your app PID 1 directly. • Shell form CMD inserts /bin/sh between Docker and your app, breaking signal delivery. • Use docker run --init or install tini for containers that spawn child processes.

CHAPTER 20

inspect • stats • exec • events • top

20.1 The Debugging Workflow

When something goes wrong with a container in production, follow this systematic approach:

- Step 1: Check logs first (docker logs -f).
- Step 2: Inspect container config and state (docker inspect).
- Step 3: Monitor resource usage (docker stats).
- Step 4: Check running processes (docker top).
- Step 5: Enter the container interactively if needed (docker exec -it sh).
- Step 6: Watch real-time events (docker events).

20.2 docker logs

```
docker logs
docker logs myapp # All logs
docker logs -f myapp # Follow in real time
docker logs --tail 50 myapp # Last 50 lines
docker logs -t myapp # With timestamps
docker logs --since 10m myapp # Last 10 minutes
```

20.3 docker inspect

```
docker inspect
# Full JSON dump of container configuration:
docker inspect myapp
# Useful filters:
docker inspect myapp | grep -i ip
docker inspect myapp | grep -A5 Mounts
docker inspect myapp | grep -A5 Env
# Go template for specific fields:
docker inspect --format='{{.State.Status}}' myapp
docker inspect --format='{{.NetworkSettings.IPAddress}}' myapp
```

20.4 docker stats

```
docker stats
# Live resource usage:
docker stats
# CONTAINER CPU % MEM USAGE / LIMIT MEM % NET I/O
# myapp 2.5% 45MiB / 512MiB 8.8% 1.2kB/2.5kB
# One-shot (no live update):
docker stats --no-stream
```

20.5 docker exec

docker exec

```
# Start an interactive shell in a running container:
docker exec -it myapp sh # sh (slim images)
docker exec -it myapp bash # bash (full images)

# Run a one-off command:
docker exec myapp env # print environment variables
docker exec myapp ps aux # list processes
docker exec myapp cat /etc/hosts
```

20.6 docker top and docker events

docker top and events

```
# List processes inside a container (like 'ps aux'):
docker top myapp

# UID PID PPID CMD
# 1001 1 0 python app.py

# Stream real-time Docker events:
docker events

# 2026-03-15T14:30:00 container start (from myapp)
# 2026-03-15T14:32:15 container die (from myapp)
# 2026-03-15T14:32:16 container start (from myapp) ← restart

# Filter events:
docker events --filter event=die
```

■ NOTE

Key Takeaways Chapter 20: • docker logs: always the first debugging step. • docker inspect: the most detailed view of container configuration and state. • docker stats: real-time CPU/memory/network monitoring. • docker exec: enter a running container without stopping it. • docker events: stream of container lifecycle events — useful for automation.

CHAPTER 21

Docker Hub • Private registries • Tagging strategy

21.1 What is an Image Registry?

An image registry is a server that stores and distributes Docker images. Instead of deploying images directly from a developer's machine, teams push images to a registry and servers pull from it. This decouples build from deploy.

Registry	Type	Use case
Docker Hub	Public / private cloud	Default, open source projects
AWS ECR	Private cloud	AWS deployments
Azure ACR	Private cloud	Azure deployments
Google Artifact Registry	Private cloud	GCP / GKE deployments
GitHub Container Registry	Private cloud	GitHub CI/CD workflows
localhost:5001 (registry:2)	Local (test)	Local development, learning

21.2 The Build → Push → Pull Workflow

Full registry workflow

```
# 1. Build the image locally:
docker build -t my-app .

# 2. Tag for the registry (registry/image:tag format):
docker tag my-app localhost:5001/my-app:v1
# OR for Docker Hub:
docker tag my-app yourusername/my-app:v1

# 3. Login to registry (required for private registries):
docker login # Docker Hub
docker login localhost:5001 # Local registry

# 4. Push:
docker push localhost:5001/my-app:v1

# 5. On another machine – pull and run:
docker pull localhost:5001/my-app:v1
docker run -p 5000:5000 localhost:5001/my-app:v1
```

21.3 Running a Local Registry

Local registry

```
# Start the official Docker registry container:
docker run -d -p 5001:5000 --name local-registry registry:2

# The registry is now at http://localhost:5001
# List all images in the registry (using the API):
curl http://localhost:5001/v2/_catalog
# {"repositories":["my-app"]}
```

21.4 Image Tagging Strategy

Tags are labels attached to images. The :latest tag is dangerous in production because it changes every time you push — running containers are NOT automatically updated when the tag moves.

Tagging strategy

```
# ■ DANGEROUS in production – latest tag can silently point to new image:
image: myapp:latest

# ■ CORRECT – use specific version tags:
image: myapp:v1.2.3
image: myapp:20260315-abcdef

# Check what image a running container is using:
docker inspect myapp | grep -i image

# "Image": "sha256:a1b2c3..."
```

21.5 Authentication Failures — ImagePullBackOff

In Kubernetes, when a pod fails to pull an image from a private registry it enters ImagePullBackOff state. The fix is to create an imagePullSecret and reference it in the pod spec.

Kubernetes imagePullSecrets

```
# Create a registry credentials secret in Kubernetes:
kubectl create secret docker-registry regcred \
--docker-username=myuser \
--docker-password=mypassword \
--docker-server=registry.example.com

# Reference it in a deployment:
spec:
  imagePullSecrets:
  - name: regcred
  containers:
  - image: registry.example.com/myapp:v1
```

■ NOTE

Key Takeaways Chapter 21: • Registries are the distribution mechanism for Docker images — never deploy from local. • Always authenticate (docker login) before pushing to private registries. • Use specific version tags (v1.2.3), never :latest in production. • Kubernetes uses imagePullSecrets to authenticate with private registries.

CHAPTER 22

Union FS • overlay2 • Copy-on-Write

Layer size trap

```
# ■ WRONG – large file persists in layer 1 even after deletion in layer 2:
RUN dd if=/dev/zero of=/tmp/bigfile.bin bs=1M count=100 # +100MB layer
RUN rm /tmp/bigfile.bin # hides file, layer unchanged
# Image size: still ~100MB larger than base

# ■ CORRECT – create and delete in the same RUN to keep it in one layer:
RUN dd if=/dev/zero of=/tmp/bigfile.bin bs=1M count=100 \
&& process_the_file \
&& rm /tmp/bigfile.bin
# Image size: no net increase
```

22.4 Inspecting Storage

Storage inspection commands

```
# Show all image layers:
docker history myimage

# Show Docker disk usage:
docker system df

# TYPE TOTAL ACTIVE SIZE RECLAIMABLE
# Images 5 3 1.2GB 800MB (66%)
# Containers 3 3 25MB 0B (0%)
# Local Volumes 2 2 500MB 0B (0%)

# Show container size (writable layer):
docker ps -s

# CONTAINER SIZE VIRTUAL SIZE
# myapp 1.5MB 121MB
# writable image + writable

# Clean up unused images/containers/volumes:
docker system prune -a --volumes
```

■ NOTE

Key Takeaways Chapter 22: • Docker uses overlay2 (Union FS) to stack read-only image layers. • Containers add a single writable layer on top — they do not modify image layers. • Copy-on-Write means modifying a file copies it from the image to the container layer. • Deleting a file in a later RUN still leaves it in an earlier layer — combine commands. • `docker system df` and `docker history` are essential for understanding image sizes.

CHAPTER ★

How every Docker concept maps to Kubernetes

B.1 Why Kubernetes?

Docker excels at running containers on a single machine. Kubernetes (K8s) is an orchestration platform that runs containers across a **cluster** of machines, handling scheduling, scaling, self-healing, rolling updates, and service discovery automatically. Every Docker concept you have learned in this book has a direct counterpart in Kubernetes.

B.2 The Full Concept Mapping

Docker Concept	Kubernetes Equivalent	Purpose
<code>docker run</code>	Pod	Smallest deployable unit
<code>docker-compose.yml</code> service	Deployment	Manages replicas & rolling updates
Container	Container (inside a Pod)	Actual application process
<code>-p host:container</code>	Service (NodePort/LoadBalancer)	Port exposure
<code>docker network</code>	NetworkPolicy	Traffic rules between pods
Named volume	PersistentVolume (PV)	Durable storage
<code>-v host:container (bind)</code>	hostPath volume	Host path mount
<code>--tmpfs</code>	emptyDir medium: Memory	RAM-backed temp storage
ENV variable	ConfigMap	Non-sensitive configuration
Docker Secret	Secret	Sensitive data
HEALTHCHECK	livenessProbe / readinessProbe	Health monitoring
<code>--memory / --cpus</code>	resources.limits	Resource constraints
USER in Dockerfile	securityContext.runAsUser	Non-root containers
<code>--read-only</code>	readOnlyRootFilesystem: true	Immutable filesystem
<code>--cap-drop ALL</code>	capabilities.drop: [ALL]	Linux capability control
seccomp profile	seccompProfile	Syscall filtering
<code>docker login</code>	imagePullSecrets	Registry authentication
Restart policy	restartPolicy	Pod restart behaviour
<code>docker build</code>	buildah / kaniko (in CI)	Image building
Image registry	Image registry (same)	Image distribution

B.3 Your First Kubernetes Deployment

Kubernetes deployment.yaml

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3 # docker run x3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myregistry/myapp:v1 # docker pull
          ports:
            - containerPort: 5000 # EXPOSE
          env:
            - name: APP_ENV
              value: production # -e APP_ENV=production
          resources:
            limits:
              memory: 256Mi # --memory=256m
              cpu: 500m # --cpus=0.5
          livenessProbe: # HEALTHCHECK
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          securityContext:
            runAsNonRoot: true # USER appuser
            readOnlyRootFilesystem: true # --read-only
            allowPrivilegeEscalation: false # --security-opt no-new-privileges
          capabilities:
            drop: [ALL] # --cap-drop ALL
```

B.4 Recommended Learning Path After This Book

- Install minikube or kind for a local Kubernetes cluster.
- Learn kubectl — the command-line tool for Kubernetes (like docker CLI).
- Deploy your first app: `kubectl create deployment myapp --image=myimage`.

- Expose it: `kubectl expose deployment myapp --port=80 --type=NodePort`.
- Scale it: `kubectl scale deployment myapp --replicas=5`.
- Study Pods, Deployments, Services, ConfigMaps, and Secrets.
- Learn about Ingress controllers for HTTP routing.
- Explore Helm for packaging Kubernetes applications.

■ TIP

Every concept you learned in this book — networking, volumes, environment variables, health checks, resource limits, security — applies directly to Kubernetes. You have already done the hard part. K8s is just Docker at scale.

Quick Reference — Essential Docker Commands

[illegible]

github.com/sandeepdass007/docker-kubernetes-learning